NPC – Capstone Project Assignment

Topic: Malware Analysis Sandbox Using VirtualBox or VMware

***

Name: Sanskriti Santoshkumar Uttam

PRN: 2125UCSF1114

Department : Cybersecurity

Course: Network Protocol Configuration (Fortinet)

Academic Year: 2026–27

---

1. Objective

The objective of this project is to understand the use of an isolated virtual machine for observing program execution, monitoring processes and network sockets, calculating file hashes, checking file reputation using VirusTotal, and recovering a clean system state using snapshots.

2. Software and Tools Used

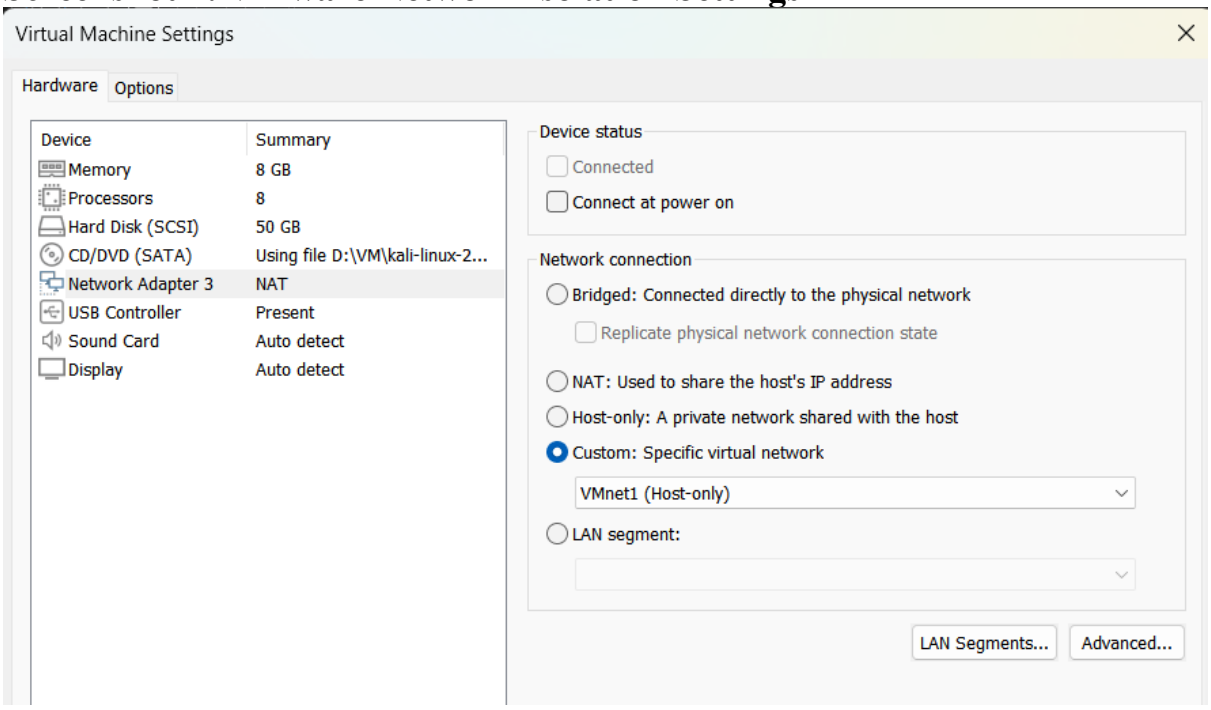
- VMware Workstation
- Kali Linux
- Python 3.13.3
- Linux terminal utilities: ps, ss, ip, and sha256sum
- VirusTotal

3. Lab Environment and Isolation

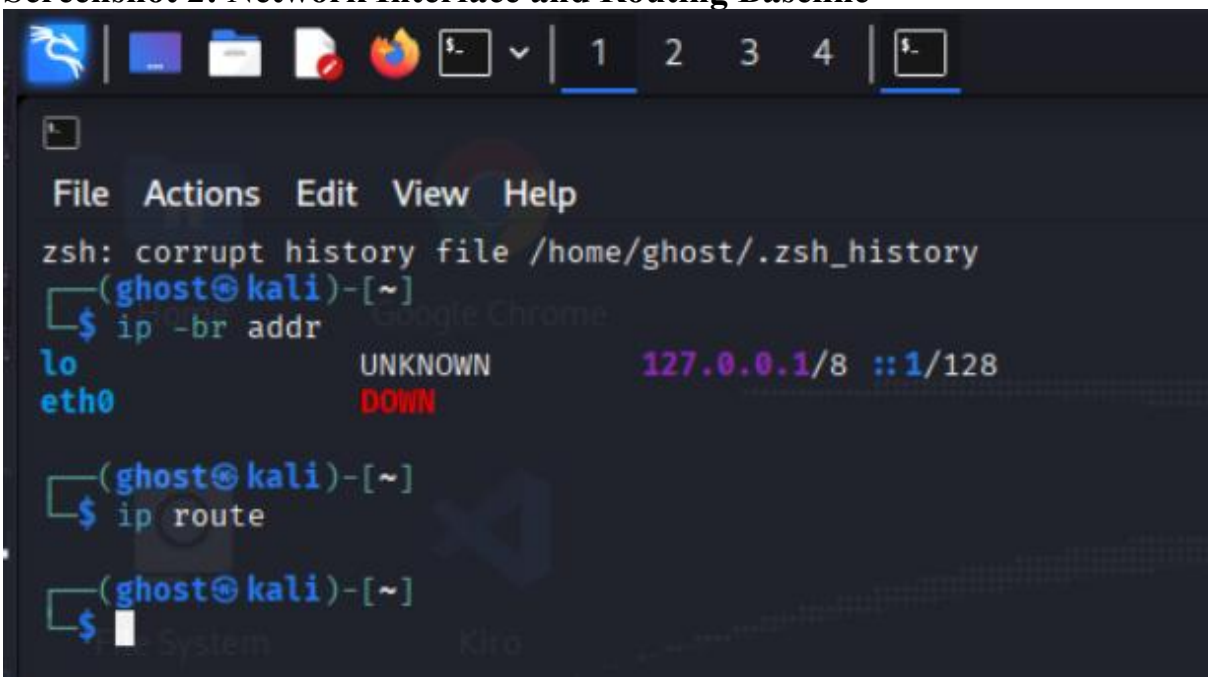
A Kali Linux virtual machine was used as the analysis environment. The virtual machine's network adapter was configured to use the host-only network VMnet1, with the adapter disconnected. The guest's network interfaces and routing table were checked to verify the disconnected state.

Shared folders were disabled, and VMware's copy-and-paste and drag-and-drop integration features were disabled after transferring the demonstration script. A clean baseline snapshot named NPC-Clean-Baseline-Final was created before the experiment.

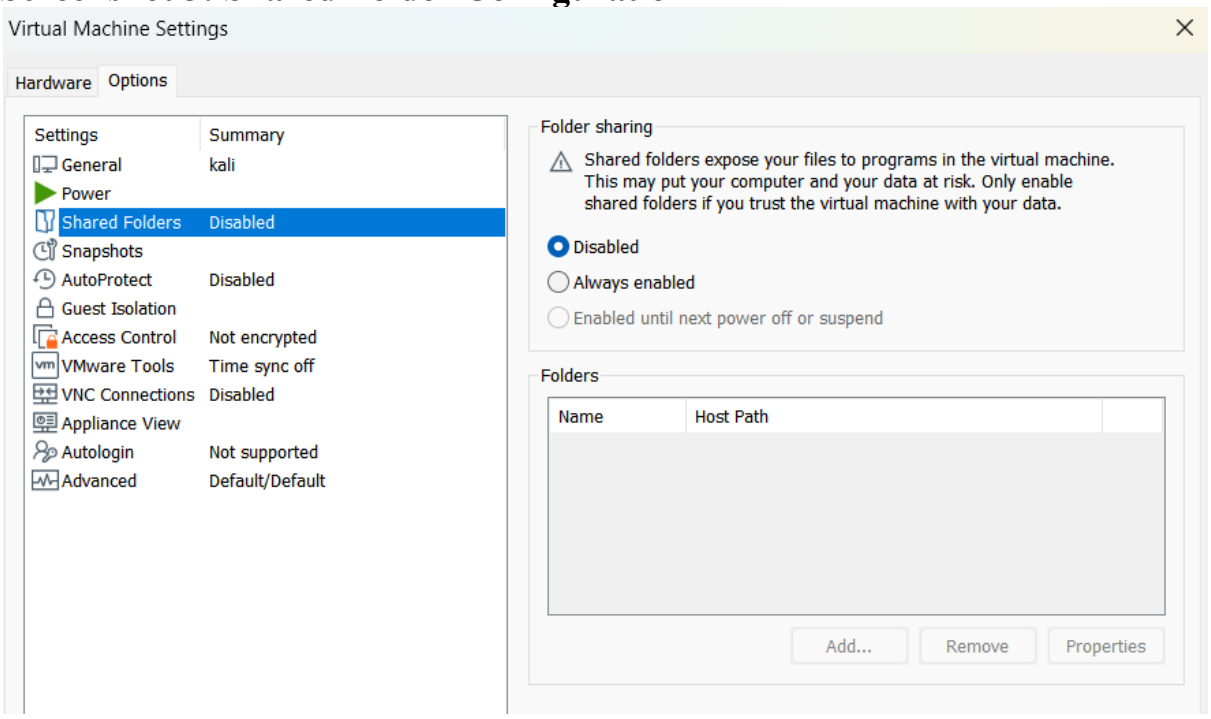
Screenshot 1: VMware Network Isolation Settings



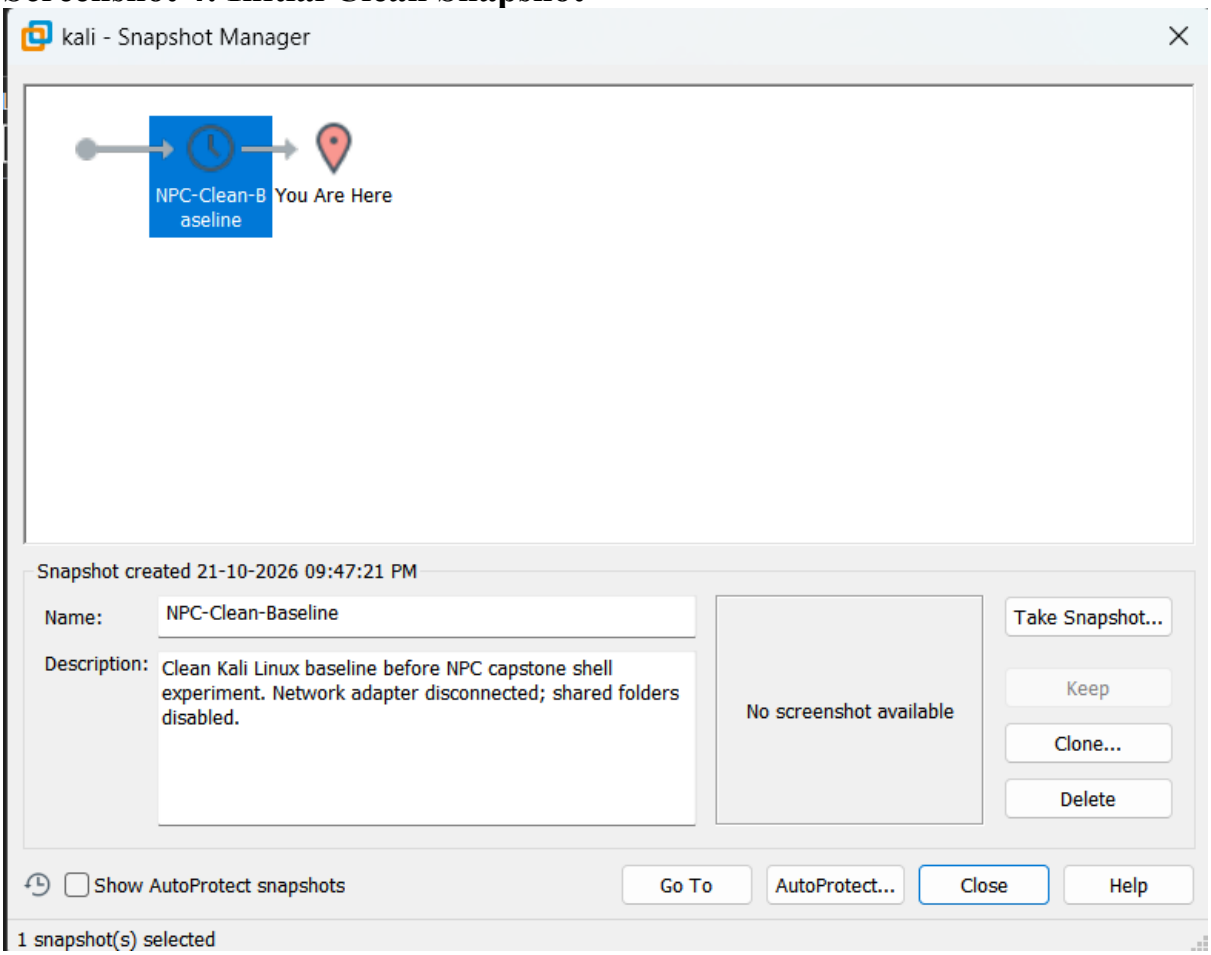
Screenshot 2: Network Interface and Routing Baseline



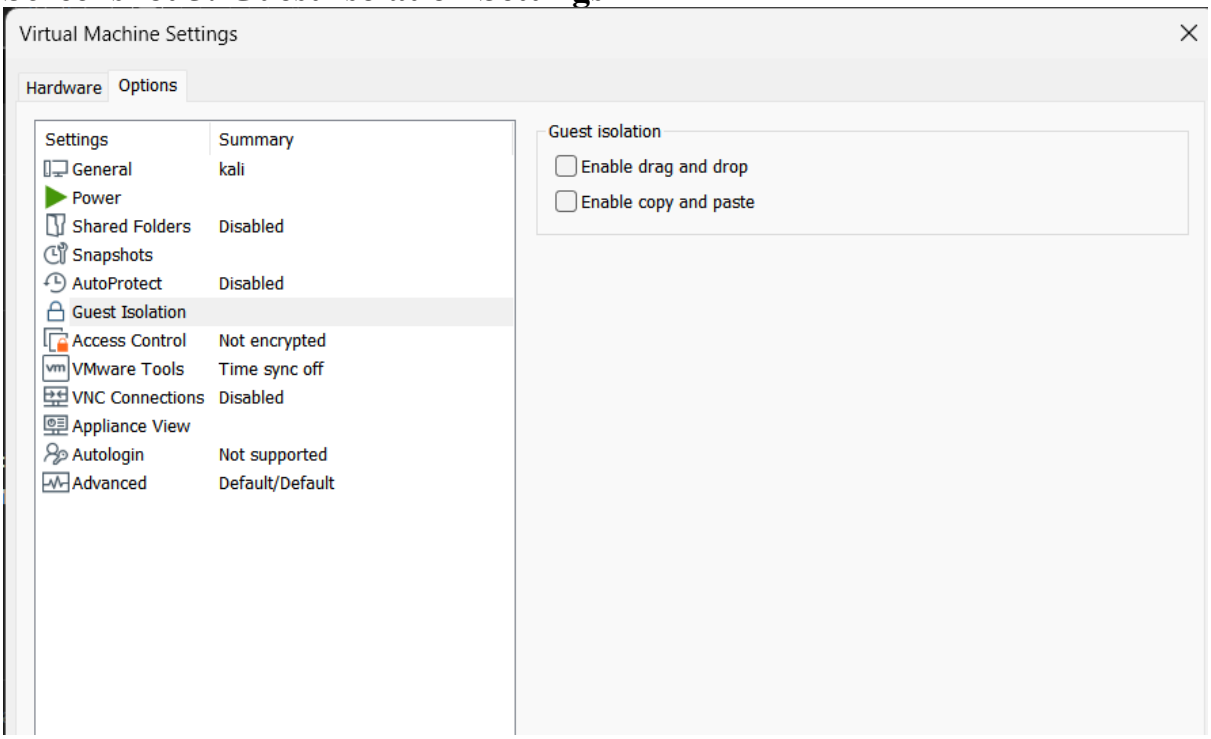
Screenshot 3: Shared Folder Configuration



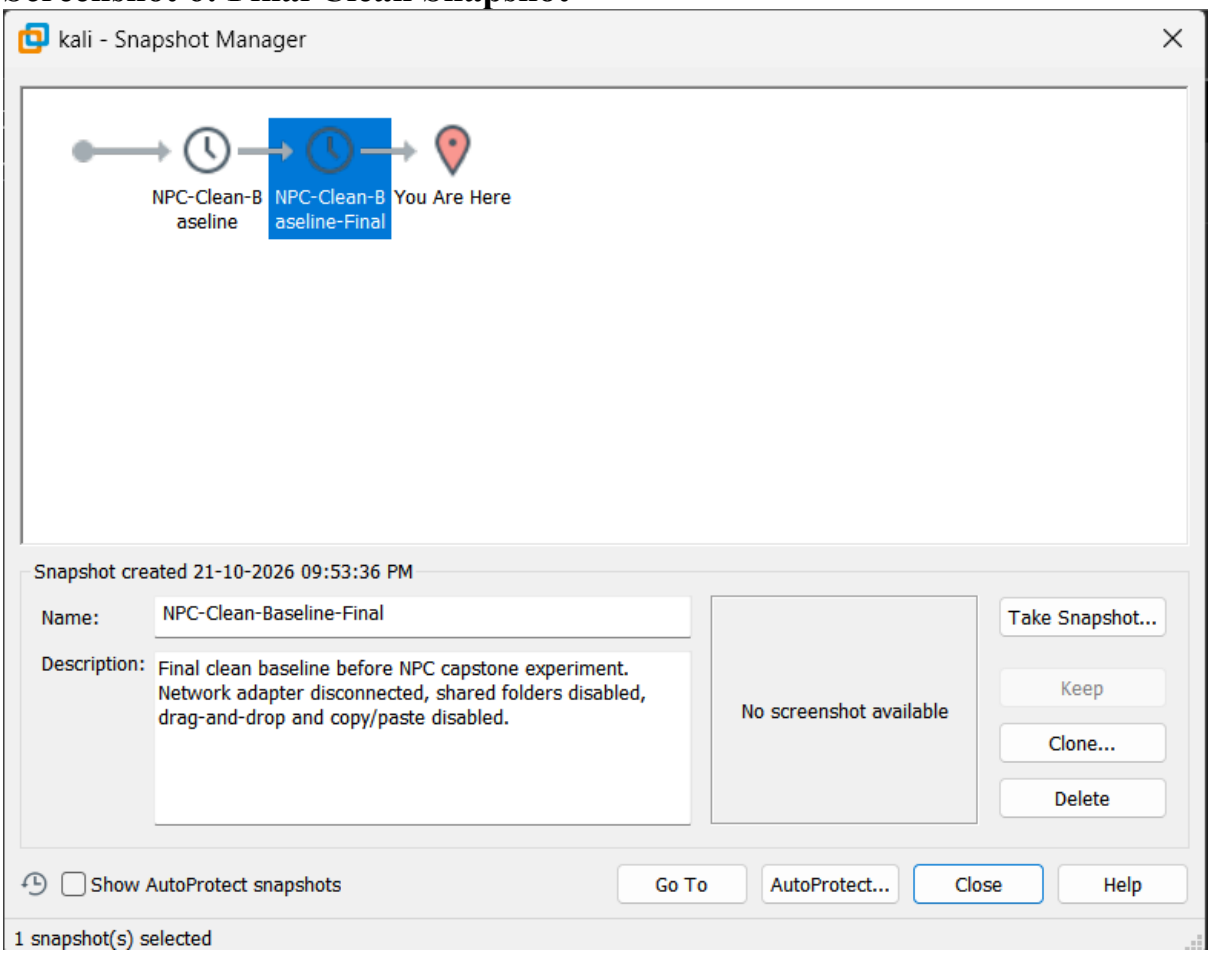
Screenshot 4: Initial Clean Snapshot



Screenshot 5: Guest Isolation Settings



Screenshot 6: Final Clean Snapshot



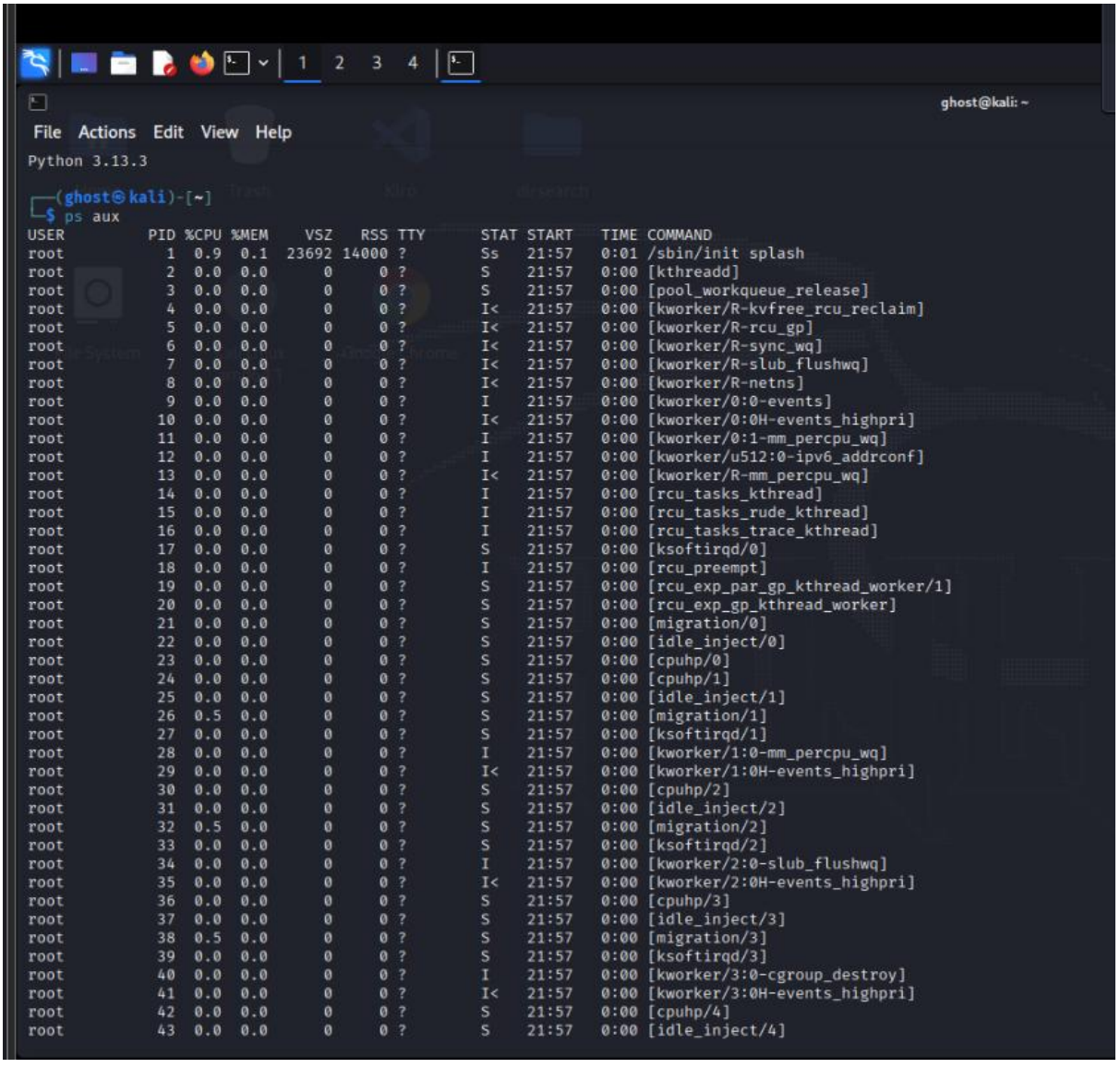
4. Baseline System Observations

Before executing the demonstration script, the existing processes and listening network sockets were inspected.

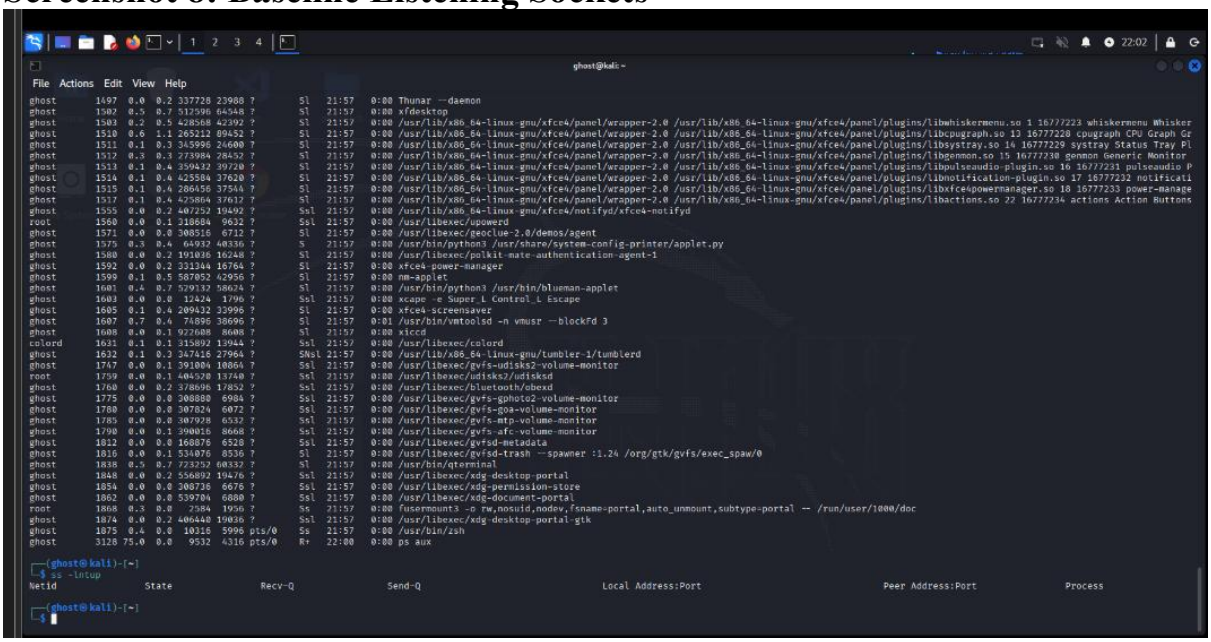
The ps aux command was used to display running processes and their resource usage. The ss -lntup command was used to inspect listening TCP and UDP sockets.

The network interface and routing checks showed that the virtual machine's Ethernet interface (eth0) was down and the routing table contained no routes.

Screenshot 7: Baseline Process List



Screenshot 8: Baseline Listening Sockets



5. Demonstration Script and Execution

A Python script named shell_demo.py was created in the directory ~/npc-capstone/.

The script displayed a demonstration message, its process ID, and its parent process ID. It then waited for 30 seconds before printing a completion message.

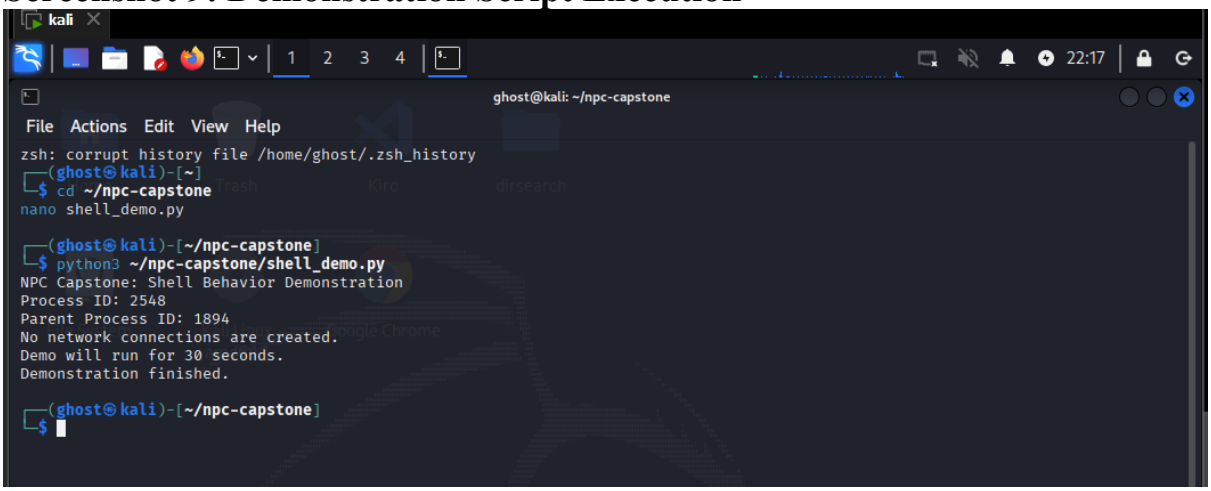
The script did not create a reverse shell, bind shell, network connection, or listening socket. It was used only to demonstrate process execution and observation in the isolated virtual machine.

The script was executed using:

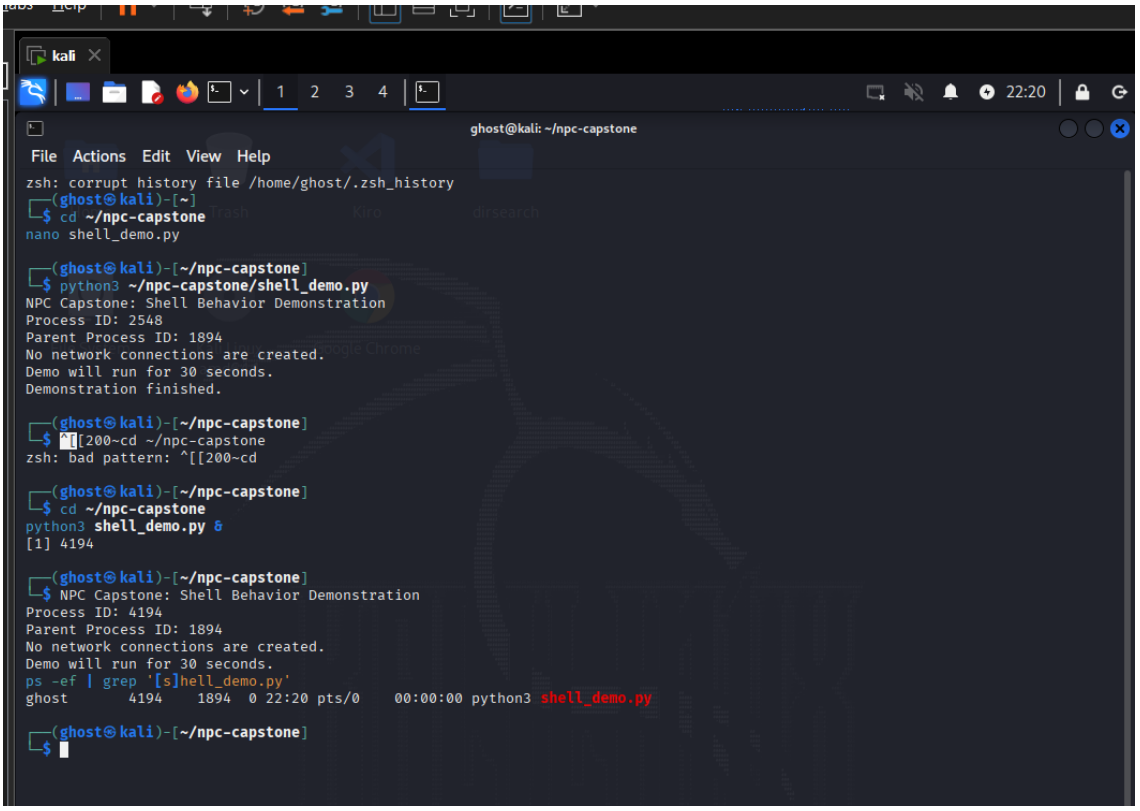
python3 ~/npc-capstone/shell_demo.py

The process was also launched in the background to allow it to be inspected while running.

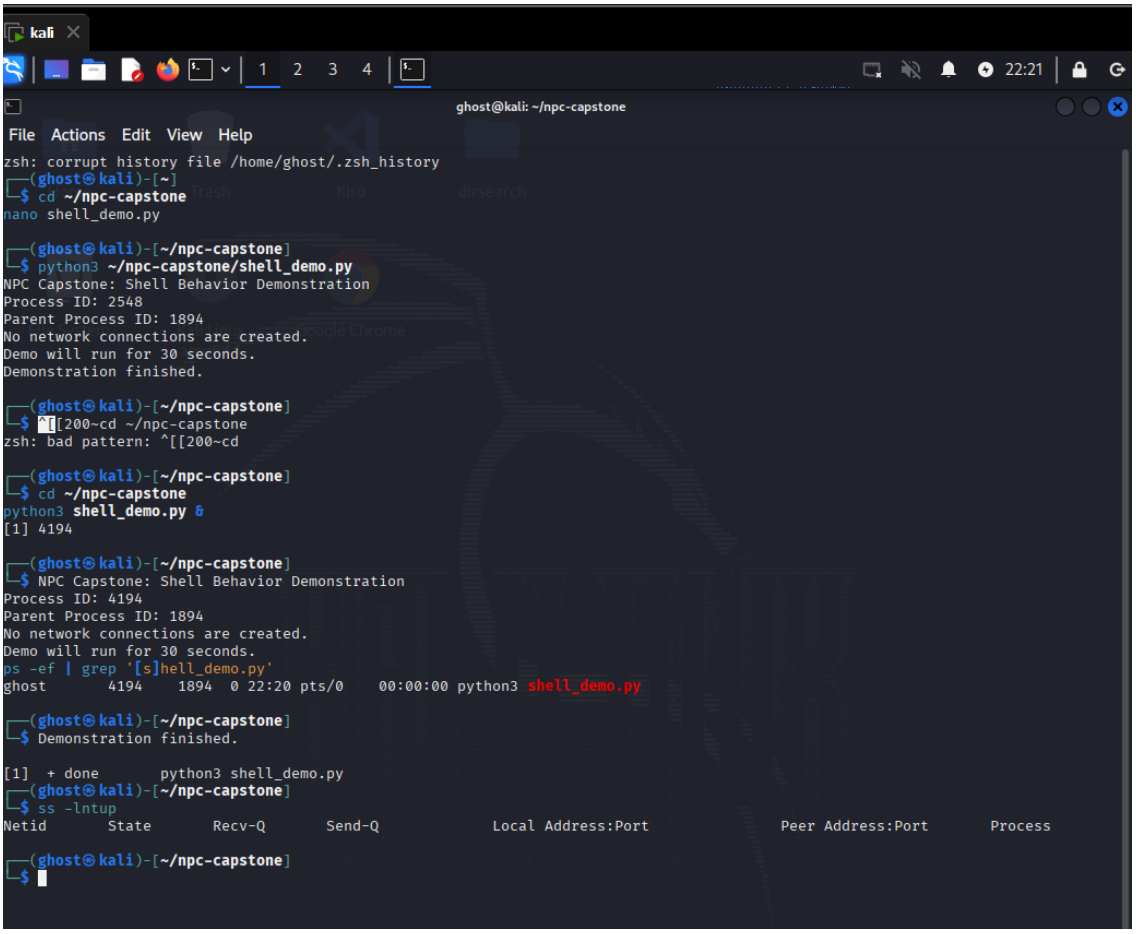
Screenshot 9: Demonstration Script Execution



Screenshot 10: Process Observation



Screenshot 11: Network Socket Observation



Observations

1. The Python script executed successfully.
2. The script displayed its process ID and parent process ID.
3. The process was visible in the process list while running.
4. The script did not intentionally establish network connections or open listening sockets.
5. The virtual machine's disconnected network configuration prevented normal network connectivity through its virtual Ethernet interface.

6. SHA-256 Hash Calculation

The SHA-256 hash of the Python script was calculated using the following command:

sha256sum shell_demo.py

The resulting hash was:

SHA-256:

e2c4cf72313ac6c3e97ee8b8f78f0dc4be34f6046a8de792bbec1c99b9ec7028d

A cryptographic hash provides a fixed-length fingerprint that can be used to identify a particular file's contents.

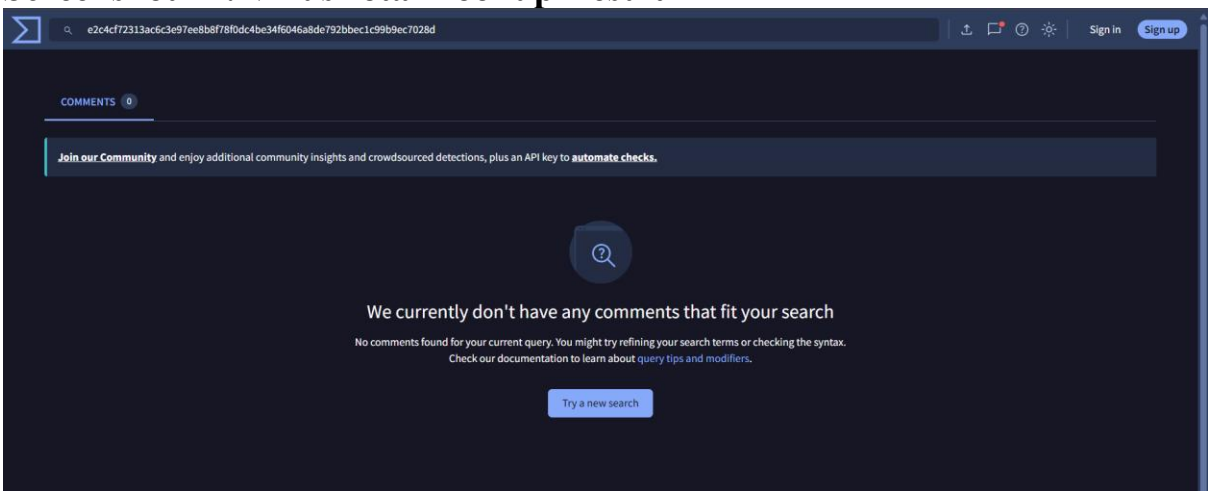
7. VirusTotal Lookup

The script's SHA-256 hash was searched on VirusTotal to check whether an existing file analysis report was available. The direct file report returned the message **"Item not found."**

No file was uploaded to VirusTotal.

Therefore, no existing detection ratio, antivirus verdict, or first-submission date was available for this hash.

Screenshot 12: VirusTotal Lookup Result



8. Snapshot Restoration and Verification

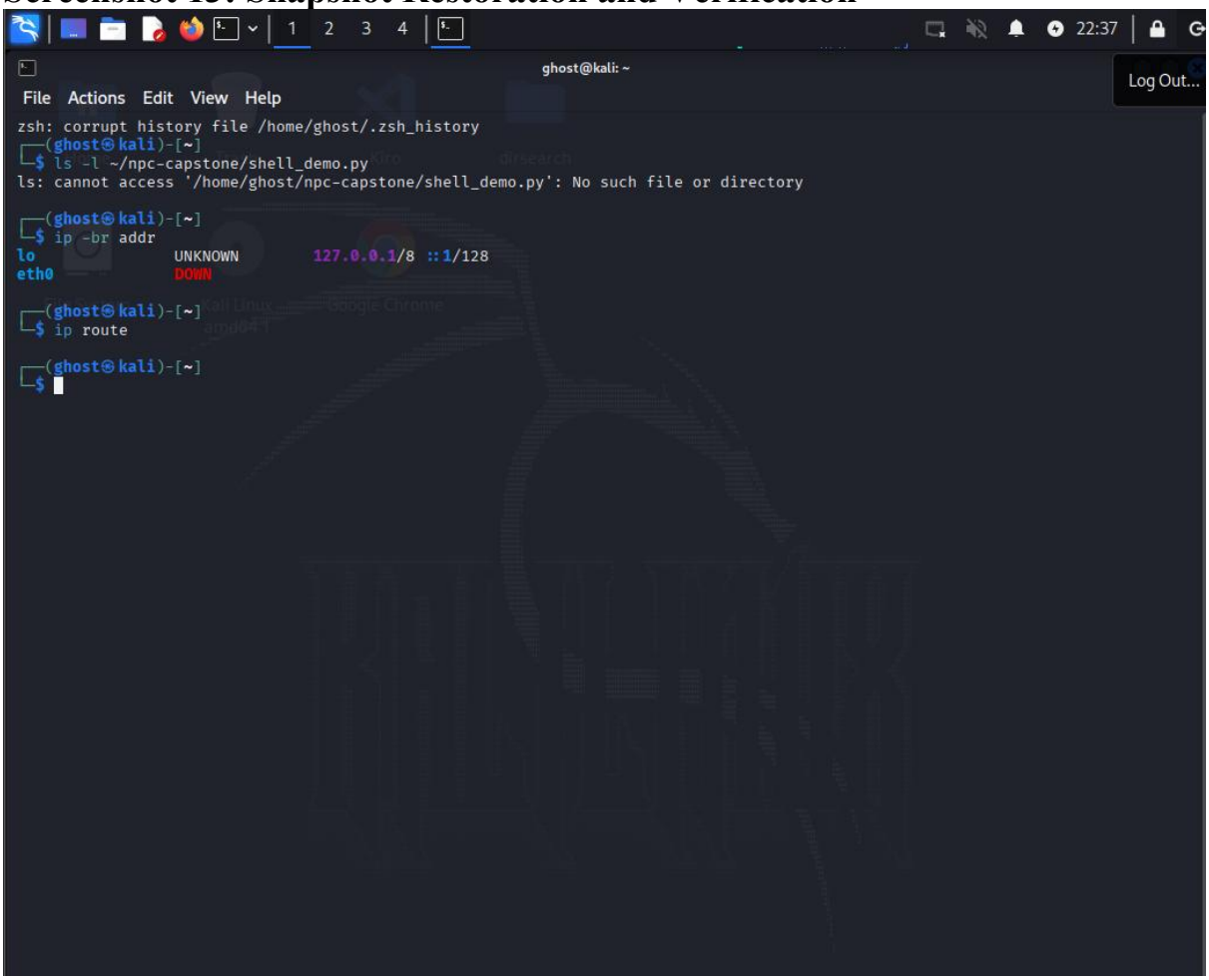
After the experiment, Kali Linux was shut down and the virtual machine was

reverted to the clean snapshot NPC-Clean-Baseline-Final using VMware Snapshot Manager.
After restoration, the following checks were performed:

- The demonstration script was checked using ls -l ~/npc-capstone/shell_demo.py.
- The network interface state was checked using ip -br addr.
- The routing table was checked using ip route.

The script was no longer present after restoration, and the network interface remained disconnected with no routes listed. These results indicated that the virtual machine had returned to the expected baseline state.

Screenshot 13: Snapshot Restoration and Verification



9. Results and Discussion

The experiment demonstrated several important sandboxing and analysis concepts:

- A virtual machine can provide a separate environment for controlled testing.
- Disabling network connectivity and shared folders reduces the risk of unintended communication or file transfer.

- Process monitoring tools can identify a running program and display its process information.
- Socket inspection tools can be used to examine listening TCP and UDP sockets.
- SHA-256 hashing can identify a file by its contents.
- VirusTotal may not have an existing report for a given hash; an unavailable report does not establish that a file is malicious or safe.
- Restoring a clean snapshot can revert changes made to the virtual machine's disk state.

Project limitation: This experiment used a benign Python process demonstration rather than a reverse shell or bind shell. Therefore, reverse-shell or bind-shell communication, associated network traffic, and related system changes were not tested. The results should not be interpreted as a complete malware-analysis or shell-behavior experiment.

10. Conclusion

This project provided practical experience with virtual-machine isolation, baseline system inspection, process monitoring, network socket inspection, file hashing, VirusTotal lookup, and snapshot recovery.

The experiment confirmed that the demonstration process could be observed while running and that the virtual machine could be restored to its clean baseline afterward. It also highlighted the importance of checking network connectivity and limiting integration between the host and guest systems when performing controlled security experiments.

The project fulfilled the process-observation, hashing, reputation-lookup, and snapshot-recovery portions of the exercise. The reverse-shell or bind-shell portion remains incomplete and would require a separate, explicitly authorized and isolated lab experiment to satisfy that requirement fully.
